



Lab 04 - Template & Display

Knowledge base - Class template

Templates in C++ with Examples - GeeksforGeeks

A Computer Science portal for geeks. It contains well written, well thought and well explained computer science

➦ <https://www.geeksforgeeks.org/templates-cpp/>

```

template <typename T>
T myMax(T x, T y)
{
    return (x > y)? x: y;
}

int main()
{
    cout << myMax<int>(3, 7) << endl;
    cout << myMax<char>('g', 'e') << endl;
    return 0;
}

// and add below code
int myMax(int x, int y)
{
    return (x > y)? x: y;
}

char myMax(char x, char y)
{
    return (x > y)? x: y;
}
    
```

Compiler internally generates and adds below code.

A. Exercises

Preparation

0. Create a solution named **Week04**
1. Create a helper class **RandomInteger** with 3 functions:

```

class RandomInteger { public: RandomInteger(); public: int next();
// [0, 65535] int next(int max); // [0, max-1] int next(int left,
right); // [left, right] };
    
```

2. Create a template class **DynamicArray / Arrray** to support storing items of any type and support automatic allocation
3. Create a template class **Result** with 4 attributes

```
template <class T> class Result { public: bool success; T data; int  
errorCode; std::string message; };
```

We use this class for general purpose, mostly in the case of returning multiple values of a function

Exercise 1: IntegersInput

Create a project named **IntegersInput**.

- Generate a random integer **n** in the range of [10, 20]
- Get **n** integers from keyboard (use **n** from previous requirement)
Each integer should be in the range of [20, 100]
- Print out all the prime numbers

Sample output

```
IntegersInput project Please enter 5 integers in the range of [20,  
100] Integer 1: 34 Integer 2: 47 Integer 3: 22 Integer 4: 55  
Integer 5: 60 Found 1 prime number(s): 47
```

Hints

To get an integer from a keyboard, you could use

```
class IntegerKeyboardProvider { public: static int next(); static  
int next(int min, int max); };
```

Advance

Alternatively, the **IntegerKeyboardProvider** could be designed like this

```
class IntegerKeyboardProvider { public: static Result<int> next();  
static Result<int> next(int min, int max); };
```

You should ask for input again if

- Input data is empty
- Invalid format of input data
- Input data is not in the required domain value

Exercise 2: FractionsProvider

Create a project named **FractionsProvider**.

- Generate a random integer n in the range of $[10, 20]$
- Get n fractions from the keyboard (use n from the previous requirement)
- Print out all the fractions in their lowest term, separated by a comma

Sample output

```
FractionsProvider project Please enter 5 fractions from the
keyboard Fraction 1: 10/6 Fraction 2: 3/4 Fraction 3: 4/1 Fraction
4: 10/2 Fraction 5: 12/6 All entered fractions: 1 2/3, 3/4, 4, 5, 2
```

Hint

To get a fraction from the keyboard, you could use

```
class FractionKeyboardProvider { public: static Fraction next(); };
```

Advance

Alternatively, the **FractionKeyboardProvider** could be designed like this

```
class FractionKeyboardProvider { public: static Result<Fraction>
next(); };
```

Exercise 3: Different display modes

Create a project named **RectanglesDisplay**

Implement all the missing pieces to see the below sample output

Main function

```
int main() { auto rectangles = RectangleProvider::getAll();
SolidRectangleConverter solid("*"); HollowRectangleConverter
hollow("-"); for (auto rectangle : rectangles) { std::cout <<
rectangle.toString() << "\n"; std::cout << solid.convert(rectangle)
<< "\n"; std::cout << hollow.convert(rectangle) << "\n"; std::cout
<< "\n"; } }
```

Sample output

```
Rectangle Width=3, Height=4 **** *  ****  ---- - - ---- Rectangle
Width=4, Height=5 ***** ***** ***** *****  ----- - - - - ----
Rectangle Width=4, Height=8 ***** ***** ***** *****  ----
----- - - - - -----
```

Hints

Given a list of **Rectangle** from the code

```
class Rectangle { private: int _width; int _height; public:
Rectangle(); Rectangle(int width, int height); std::string
toString(); }; class RectangleProvider { public: static
std::vector<Rectangle> getAll(); }; std::vector<Rectangle>
RectangleProvider::getAll() { return { // Hard code at this week is
enough Rectangle(3, 4), Rectangle(4, 5), Rectangle(4, 8) }; }
```

The converters have prototypes like this

```
class SolidRectangleConverter { public: std::string
convert(Rectangle info); }; class HollowRectangleConverter {
public: std::string convert(Rectangle info); };
```

Exercise 4: Table display

Name your project **StudentsTable**

Main function

```
int main() { auto students = StudentProvider::getAll();
std::vector<std::string> headers = { "STT", "MSSV", "Ho va ten",
"Email" }; std::vector<int> columnSizes = {3, 4, 20, 20};
StudentsToTableConverter table(headers, columnSizes); auto info =
table.convert(students); std::cout << info; return 0; }
```

Sample output

```
|STT|MSSV| Ho va ten| Email| -----
----- | 1| 001| Nguyen Van Minh| nvminh@gmail.com| | 2|
002| Cao Ha Thang| chthang@gmail.com| | 3| 003| Tran Duc Tri|
tdtri@gmail.com|
```

Hints

You can hard code your data access object like this

```
class Student { public: std::string _id; std::string _fullname;
std::string _email; public: Student(); Student(std::string id,
std::string fullname, std::string email); }; class StudentProvider
{ public: static std::vector<Student> getAll(); };
std::vector<Student> StudentProvider::getAll() { return {
Student("001", "Nguyen Van Minh", "nvminh@gmail.com"),
Student("002", "Cao Ha Thang", "chthang@gmail.com"), Student("003",
"Tran Duc Tri", "tdtri@gmail.com") }; }
```

To convert a vector of students into a table, you can code like this

```
class StudentsToTableConverter { private: std::vector<std::string>
_headers; std::vector<int> _columnSizes; public:
StudentsToTableConverter();
StudentsToTableConverter(std::vector<std::string> headers,
std::vector<int> columnSizes); std::string
convert(std::vector<Student> students); private: // Inner class
class StudentToRowConverter { private: std::vector<int>
_columnSizes; public: StudentToRowConverter();
StudentToRowConverter(std::vector<int> columnSizes); std::string
convert(Student student); }; };
```

Please notice that the table has extra columns, which is "STT" - the order number
Before using the class `TableConverter`, we must do some configuration like this

```
std::vector<std::string> headers = { "STT", "MSSV", "Ho va ten",
"Email" }; std::vector<int> columnSizes = {3, 4, 20, 20};
StudentsToTableConverter table(headers, columnSizes);
```

B. Submission instructions

1. Delete the hidden folder `.vs`
2. Compress into the format {StudentID}.zip/.rar and submit back this file.
 - For example if your student id is 1812445, your filename should be 1812445.zip or 1812445.rar

C. Frequently asked questions

No question yet.

